Git and GitHub

`init` : Getting started

Let's take your first few steps in your Git journey.

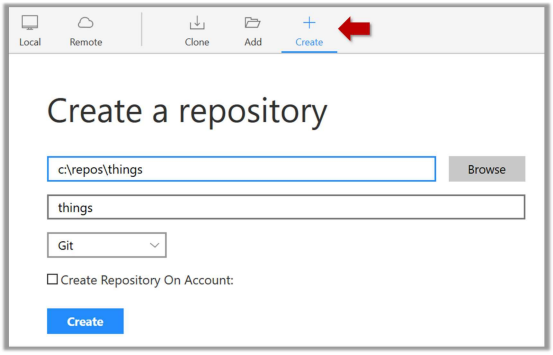
1. First, install Sourcetree (installation instructions), which is Git + a GUI for Git. If you prefer to use Git via the command line (i.e., without a GUI), you can install Git instead.

2. Next, create a directory for the repo (e.g., a directory named `things`).

3. Then, initialize a repository in that directory.

Sourcetree

Windows: Click `File` → `Clone/New...` → Click on `+ Create` button on the top menu bar.



Enter the location of the directory and click `Create`.

Mac: `New...` → `Create Local Repository` (or `Create New Repository`) → Click `...` button to select the folder location for the repository → click the `Create` button.

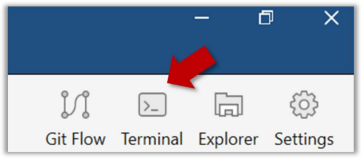
Go to the `things` folder and observe how a hidden folder `.git` has been created.

Windows: To see the hidden folders, you might have to configure Windows Explorer to show hidden files first.

CLI

Open a Git Bash Terminal.

If you installed Sourcetree, you can click the `Terminal` button to open a GitBash terminal (on a Linux/Mac environment, even a regular terminal should do).



Navigate to the `things` directory.
Use the command `git init` which should initialize the repo.

```
$ cd /c/repos/things
```

```
$ git init
```

↓

```
Initialized empty Git repository in c:/repos/things/.git/
```

You can use the *list all* command `ls -a` to view all files, which should show the `.git` directory that was created by the previous command.

```
$ ls -a
.  ..  .git
```

You can also use the `git status` command to check the status of the newly-created repo. It should respond with something like the following:

```
$ git status
```

↓

```
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

As you see above, this textbook explains how to use Git via Sourcetree (a GUI client) as well as via the Git CLI. If you are new to Git, **we recommend you learn <u>both</u> the GUI method and the CLI method** -- The GUI method will help you visualize the result better while the CLI method is more universal (i.e., you will not be tied to any GUI) and more flexible/powerful.

It is fine to learn the CLI way only (using Sourcetree is optional), especially if you normally prefer to work with CLI over GUI.

`commit` : Saving changes to history

After initializing a repository, Git can help you with revision controlling files inside the *working directory*. However, it is not automatic. You need to tell Git which of your changes (aka *revisions*) should be *committed* to its memory for later use. Saving changes into Git's memory in that way is called *committing* and a change saved to the revision history is called a *commit*.

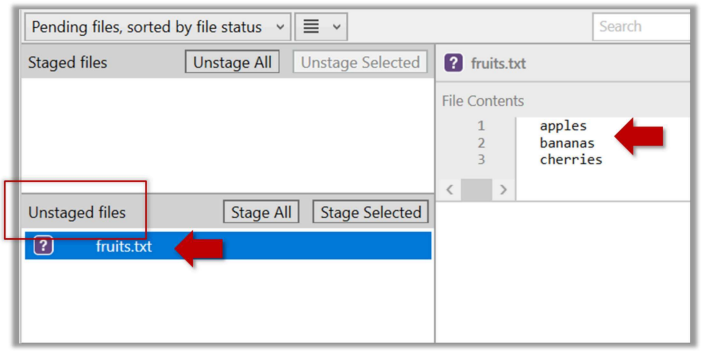
Here are the steps you can follow to learn how to create Git commits:

1. Do some changes to the content inside the *working directory* e.g., create a file named `fruits.txt` in the `things` directory and add some dummy text to it.

2. Observe how the file is detected by Git.

Sourcetree

The file is shown as 'unstaged'.



CLI

You can use the `git status` command to check the status of the working directory.

```
$ git status
```

↓

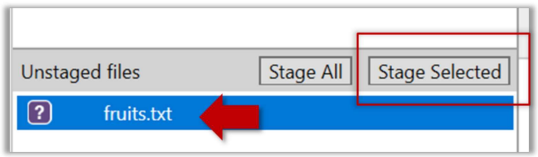
```
# On branch master
#
# No commits yet
#
# Untracked files:
#   (use "git add <file>..." to include in what will be committed)
#
#   a.txt
nothing added to commit but untracked files present (use "git add" to track)
```

3. Stage the changes to commit: Although Git has detected the file in the working directory, it will not do anything with the file unless you tell it to. Suppose you want to commit the current changes to the file. First, you should *stage* the file, which is how you tell Git which changes you want to include in the next commit.

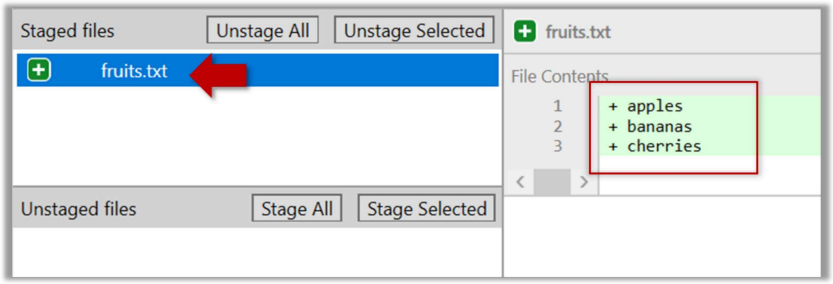
ℹ When you stage a change, the change is moved to the *staging area*, which is a file Git uses to store information about what will go into your next commit. **The staging area is also called the 'index'** by the Git practitioners.

Sourcetree

Select the `fruits.txt` and click on the `Stage Selected` button.



`fruits.txt` should appear in the `Staged files` panel now.



ℹ **If Sourcetree shows a `\ No newline at the end of the file` message** below the staged lines (i.e., below the `cherries` line in the above screenshot), that is because you did not hit `enter` after entering the last line of the file (hence, Git is not sure if that line is complete). To rectify, move the cursor to the end of the last line in that file and hit `enter` (like you are adding a blank line below it). This new change will now appear as an 'unstaged' change. Stage it as well.

CLI

You can use the `stage` or the `add` command (they are synonyms, `add` is the more popular choice) to stage files.

```
$ git add fruits.txt
$ git status
```

↓

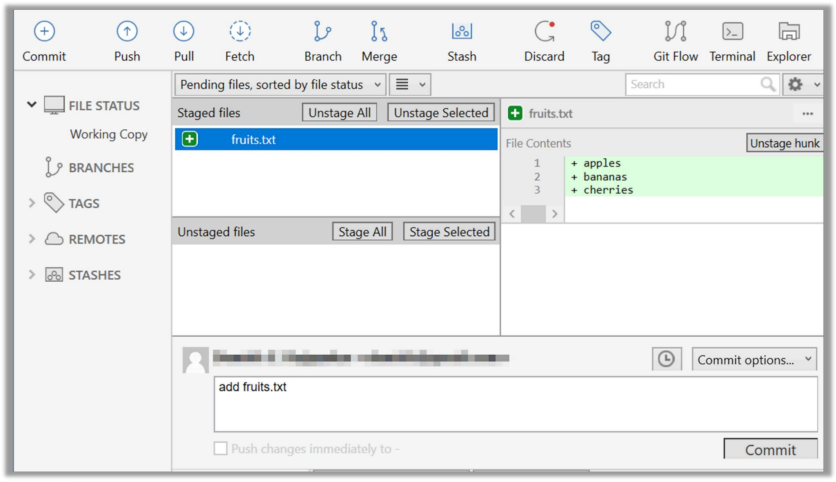
```
# On branch master
#
# No commits yet
#
# Changes to be committed:
#   (use "git rm --cached <file>..." to unstage)
#
#       new file:   fruits.txt
#
```

---

4. Commit the staged version of `fruits.txt` .

Sourcetree

Click the `Commit` button, enter a commit message e.g. `add fruits.txt` into the text box, and click `Commit` .



CLI

Use the `commit` command to commit. The `-m` switch is used to specify the commit message.

```
$ git commit -m "Add fruits.txt"
```

You can use the `log` command to see the commit history.

```
$ git log
```

↓

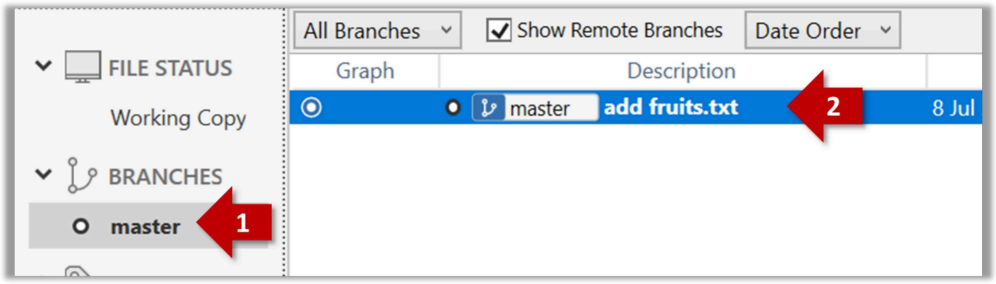
```
commit 8fd30a6910efb28bb258cd01be93e481caeab846
Author: … < … @... >
Date:   Wed Jul 5 16:06:28 2017 +0800

  Add fruits.txt
```

Note the existence of something called the `master` branch. Git uses a mechanism called 'branches' to facilitate evolving file content in parallel (we'll learn git branching in a later topic). Furthermore, Git auto-creates a branch named `master` (or `main`) on which the commits go on by default.

Sourcetree

Expand the `BRANCHES` menu and click on the `master` to view the history graph, which contains only one node at the moment, representing the commit you just added. Also note a label `master` attached to the commit.

This label points to the latest commit on the `master` branch.

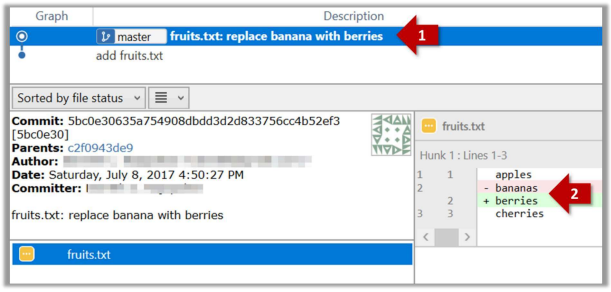


CLI

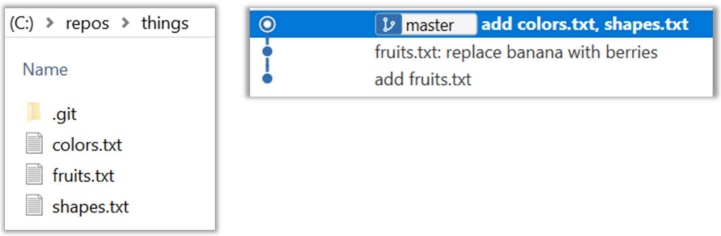
Run the `git status` command and note how the output contains the phrase `on branch master`.

5. Do a few more commits.

1. Make some changes to `fruits.txt` (e.g. add some text and delete some text). Stage the changes, and commit the changes using the same steps you followed before. You should end up with something like this.



2. Next, add two more files `colors.txt` and `shapes.txt` to the same working directory. Add a third commit to record the current state of the working directory.

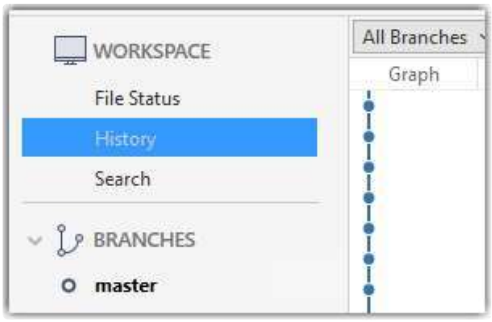


> **You can decide what to stage and what to leave unstaged.** When staging changes to commit, you can leave some files unstaged, if you wish to not include them in the next commit. In fact, Git even allows some changes in a file to be staged, while other changes in the same file to be unstaged. This flexibility is particularly useful when you want to put all related changes into a commit while leaving out unrelated changes.

6. See the revision graph: Note how commits form a path-like structure aka the *revision tree/graph*. In the revision graph, each commit is shown as linked to its 'parent' commit (i.e., the commit before it).

Sourcetree

To see the revision graph, click on the `History` item (listed under the `WORKSPACE` section) on the menu on the right edge of Sourcetree.



CLI

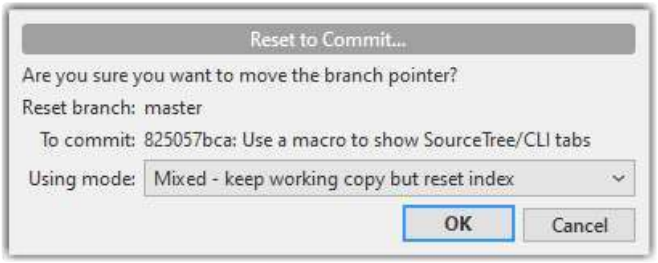
The `gitk` command opens a rudimentary graphical view of the revision graph.

How to undo/delete a commit?

Sourcetree

To undo the last commit, right-click on the commit just before it, and choose `Reset current branch to this commit` .

In the next dialog, choose the mode `Mixed - keep working copy but reset index` option. This will make the offending commit disappear but will keep the changes that you included in that commit intact.



If you use the `Soft - ...` mode instead, the last commit will be undone as before, but the changes included in that commit will stay in the staging area.

To delete the last commit entirely (i.e., undo the commit and also discard the changes included in that commit), do as above but choose the `Hard - ...` mode instead.

To undo/delete last n commits, right-click on the commit just before the last n commits, and do as above.

CLI

To undo the last commit, but keep the changes in the staging area, use the following command.

```
$ git reset --soft HEAD~1
```

To undo the last commit, and remove the changes from the staging area (but not discard the changes), used `--mixed` instead of `--soft` .

```
$ git reset --mixed HEAD~1
```

To delete the last commit entirely (i.e., undo the commit and also discard the changes included in that commit), do as above but use the `--hard` flag instead (i.e., do a hard reset).

```
$ git reset --hard HEAD~1
```

To undo/delete last n commits: `HEAD~1` is used to tell Git you are targeting the commit one position before the latest commit -- in this case the target commit is the one we want to reset to, not the one we want to undo (as the command used is `reset`). To undo/delete two last commits, you can use `HEAD~2` , and so on.

Omitting files from revision control ⚡⚡

Often, there are files inside the Git working folder that you don't want to revision-control e.g., temporary log files. Follow the steps below to learn how to configure Git to ignore such files.

1. Add a file into your repo's working folder that you supposedly don't want to revision-control e.g., a file named `temp.txt` . Observe how Git has detected the new file.

2. Configure Git to ignore that file:

Sourcetree

The file should be currently listed under `Unstaged files` . Right-click it and choose `Ignore…` . Choose `Ignore exact filename(s)` and click `OK` .

Observe that a file named `.gitignore` has been created in the working directory root and has the following line in it.

.gitignore

```
1 temp.txt
```

CLI

Create a file named `.gitignore` in the working directory root and add the following line in it.

.gitignore

```
temp.txt
```

The `.gitignore` file

The `.gitignore` file tells Git which files to ignore when tracking revision history. That file itself can be either revision controlled or ignored.

- To version control it (the more common choice – which allows you to track how the `.gitignore` file changes over time), simply commit it as you would commit any other file.
- To ignore it, follow the same steps you followed above when you set Git to ignore the `temp.txt` file.
- It supports file patterns e.g., adding `temp/*.tmp` to the `.gitignore` file prevents Git from tracking any `.tmp` files in the `temp` directory.

More information about the `.gitignore` file: git-scm.com/docs/gitignore

Files recommended to be omitted from version control

- **Binary files** *generated* when building your project e.g., `*.class`, `*.jar`, `*.exe` (reasons: 1. no need to version control these files as they can be generated again from the source code 2. Revision control systems are optimized for tracking text-based files, not binary files.
- **Temporary files** e.g., log files generated while testing the product
- **Local files** i.e., files specific to your own computer e.g., local settings of your IDE
- **Sensitive content** i.e., files containing sensitive/personal information e.g., credential files, personal identification data (especially, if there is a possibility of those files getting leaked via the revision control system).

`tag` : Naming commits ⚡⚡⚡

Each Git commit is uniquely identified by a hash e.g., `d670460b4b4aece5915caf5c68d12f560a9fe3e4`. As you can imagine, using such an identifier is not very convenient for our day-to-day use. As a solution, Git allows adding a more human-readable *tag* to a commit e.g., `v1.0-beta`.

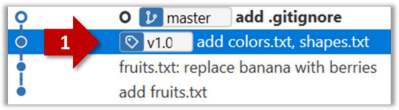
Here's how you can tag a commit in a local repo:

Sourcetree

Right-click on the commit (in the graphical revision graph) you want to tag and choose `Tag...`.

Specify the tag name e.g. `v1.0` and click `Add Tag`.

The added tag will appear in the revision graph view.



CLI

To add a tag to the current commit as `v1.0`:

```
$ git tag v1.0
```

To view tags:

```
$ git tag
```

To learn how to add a tag to a past commit, go to the 'Git Basics – Tagging' page of the git-scm book and refer the 'Tagging Later' section.

After adding a tag to a commit, you can use the tag to refer to that commit, as an alternative to using the hash.

ℹ **Annotated vs Lightweight Tags**: The Git tags explained above are known as *lightweight* tags. There is another type of Git tags called *annotated* tags. See git-scm.com/book for more info.

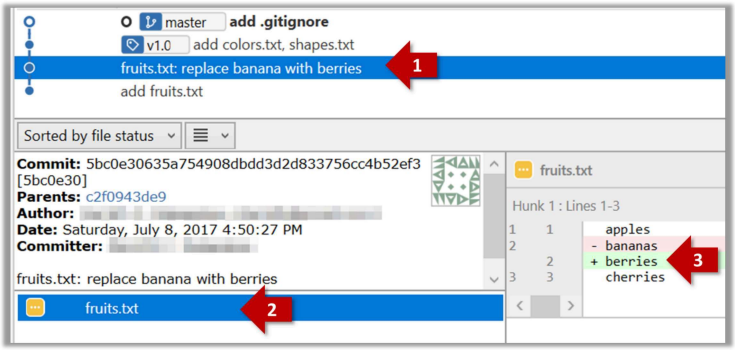
! **Tags are different from commit messages**, in purpose and in form. A commit message is a description of the commit that is *part of* the commit itself. A tags is a short name for a commit, which exists as a separate entity that *points to* a commit.

`diff` : Comparing revisions ⚡⚡

Git can show you what changed in each commit.

Sourcetree

To see which files changed in a commit, click on the commit. To see what changed in a specific file in that commit, click on the file name.



CLI

```
$ git show < part-of-commit-hash >
```

Example:

```
$ git show 5bc0e306
```

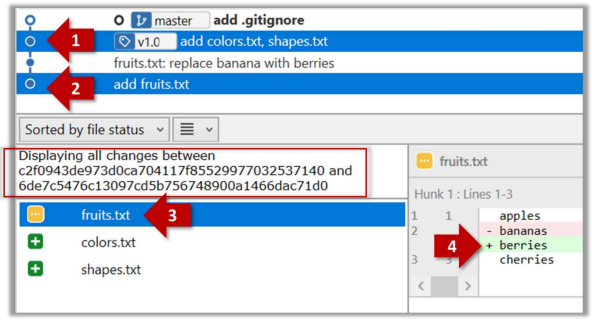
↓

```
commit 5bc0e30635a754908dbdd3d2d833756cc4b52ef3
Author: … < … >
Date:   Sat Jul 8 16:50:27 2017 +0800

    fruits.txt: replace banana with berries

diff --git a/fruits.txt b/fruits.txt
index 15b57f7..17f4528 100644
--- a/fruits.txt
+++ b/fruits.txt
@@ -1,3 +1,3 @@
 apples
-bananas
+berries
 cherries
```

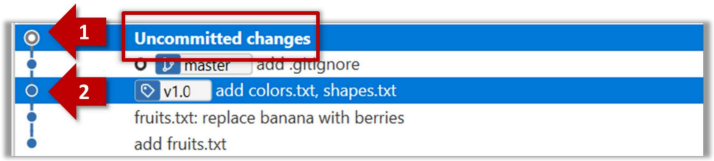
Git can also show you the difference between two points in the history of the repo.

Sourcetree

Select the two points you want to compare using `Ctrl` + `Click` . The differences between the two selected versions will show up in the bottom half of Sourcetree, as shown in the screenshot below.



The same method can be used to compare the current state of the working directory (which might have uncommitted changes) to a point in the history.



CLI

The `diff` command can be used to view the differences between two points of the history.

- `git diff` : shows the changes (uncommitted) since the last commit.
- `git diff 0023cdd..fcd6199` : shows the changes between the points indicated by commit hashes.
  Note that when using a commit hash in a Git command, you can use only the first few characters (e.g., first 7-10 chars) as that's usually enough for Git to locate the commit.
- `git diff v1.0..HEAD` : shows changes that happened from the commit tagged as `v1.0` to the most recent commit.

Resources

- Git-Tower Tutorial: Inspecting Changes with Diffs
- How to view the next page of a `git` command result

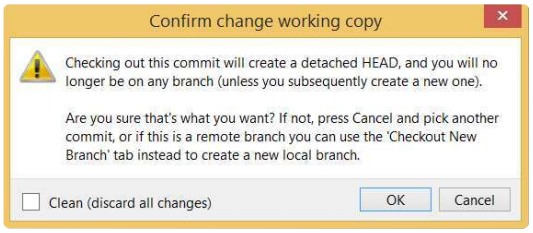
`checkout` : Retrieving a specific revision ⚡⚡

Git can load a specific version of the history to the working directory. Note that if you have uncommitted changes in the working directory, you need to stash them first to prevent them from being overwritten.

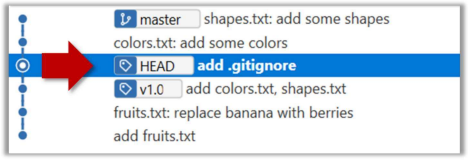
Sourcetree

Double-click the commit you want to load to the working directory, or right-click on that commit and choose `Checkout...`.

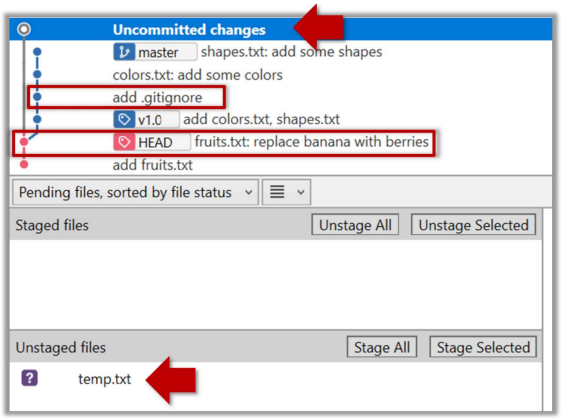
Click `OK` to the warning about 'detached HEAD' (similar to below).



The specified version is now loaded to the working folder, as indicated by the `HEAD` label. `HEAD` is a reference to the currently checked out commit.



If you checkout a commit that comes before the commit in which you added the `.gitignore` file, Git will now show ignored files as 'unstaged modifications' because at that point Git hasn't been told to ignore those files.



To go back to the latest commit, double-click it.

CLI

Use the `checkout <commit-identifier>` command to change the working directory to the state it was in at a specific past commit.

- `git checkout v1.0` : loads the state as at commit tagged `v1.0`
- `git checkout 0023cdd` : loads the state as at commit with the hash `0023cdd`
- `git checkout HEAD~2` : loads the state that is 2 commits behind the most recent commit

For now, you can ignore the warning about 'detached HEAD'.

If you checkout a commit that comes before the commit in which you added the `.gitignore` file, Git will now show ignored files as 'unstaged modifications' because at that point Git hasn't been told to ignore those files.

`clone` : Copying a repo

Given below is an example scenario you can try yourself to learn Git cloning.

Suppose you want to clone the sample repo samplerepo-things to your computer.

! Note that the URL of the GitHub project is different from the URL you need to clone a repo in that GitHub project. e.g.

GitHub project URL: `https://github.com/se-edu/samplerepo-things`
Git repo URL: `https://github.com/se-edu/samplerepo-things.git` (note the `.git` at the end)

Sourcetree

`File` → `Clone / New…` and provide the URL of the repo and the destination directory.

CLI

You can use the `clone` command to clone a repo.

Follow the instructions given here.

`pull` , `fetch` : Downloading data from other repos

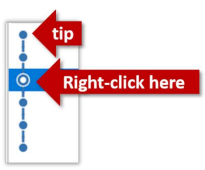
Here's a scenario you can try in order to learn how to pull commits from another repo to yours.

1. Clone a repo (e.g., the repo used in [Git & GitHub → Clone]) to be used for this activity.

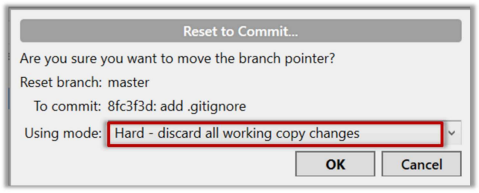
2. Delete the last few commits to simulate cloning the repo a few commits ago.

Sourcetree

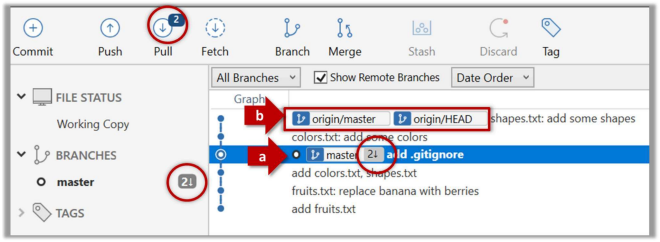
Right-click the target commit (i.e. the commit that is 2 commits behind the tip) and choose `Reset current branch to this commit` .



Choose the `Hard - …` option and click `OK` .



This is what you will see.



Note the following (cross-refer the screenshot above):

- Arrow marked as `a` : The local repo is now at this commit, marked by the `master` label.
- Arrow marked as `b` : The `origin/master` label shows what is the latest commit in the `master` branch in the remote repo. `origin` is the default name given to the upstream repo you cloned from. You can ignore the `origin/HEAD` label for now.

CLI

Use the `reset` command to delete commits at the tip of the revision history.

```
$ git reset --hard HEAD~2
```

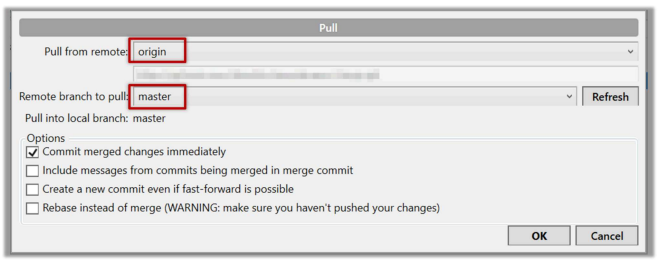
More info on the `git reset` command can be found here.

Now, your local repo state is exactly how it would be if you had cloned the repo 2 commits ago, as if somebody has added two more commits to the remote repo since you cloned it.

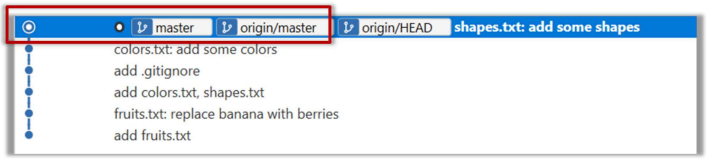
3. Pull from the remote repo: To get those missing commits to your local repo (i.e. to sync your local repo with upstream repo) you can do a pull.

Sourcetree

Click the `Pull` button in the main menu, choose `origin` and `master` in the next dialog, and click `OK` .



Now you should see something like this where `master` and `origin/master` are both pointing the same commit.



CLI

```
$ git pull origin
```

ℹ **You can also do a `fetch` instead of a `pull`** in which case the new commits will be downloaded to your repo but the working directory will remain at the current commit. To move the current state to the latest commit that was downloaded, you need to do a `merge`. A `pull` is a shortcut that does both those steps in one go.

Working with multiple remotes

When you clone a repo, Git automatically adds a *remote* repo named `origin` to your repo configuration. As you know, you can pull commits from that repo. Furthermore, a Git repo can work with remote repos other than the one it was cloned from.

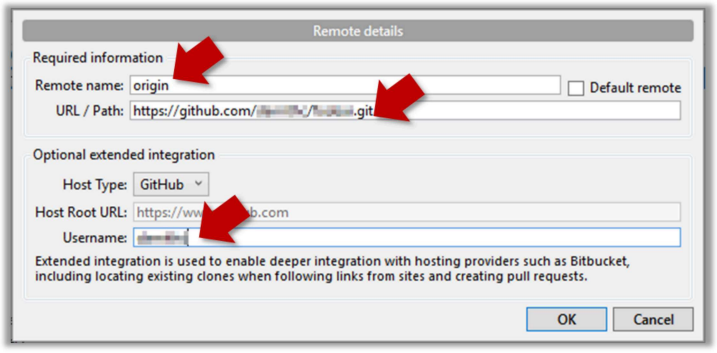
To communicate with another remote repo, you can first add it as a *remote* of your repo. Here is an example scenario you can follow to learn how to pull from another repo:

Sourcetree

1. Open the local repo in Sourcetree. Suggested: Use your local clone of the `samplerepo-things` repo.

2. Choose `Repository` → `Repository Settings` menu option.

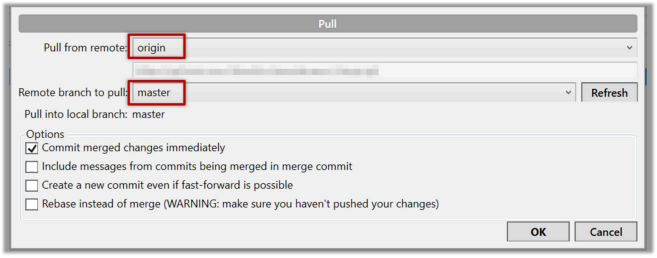
3. Add a new *remote* to the repo with the following values.



   - `Remote name` : the name you want to assign to the remote repo e.g., `upstream1`
   - `URL/path` : the URL of your repo (ending in `.git`) that. Suggested: `https://github.com/se-edu/samplerepo-things-2.git` (`samplerepo-things-2` is another repo that has a shared history with `samplerepo-things`)
   - `Username` : your GitHub username

4. Now, you can fetch or pull (pulling will fetch the branch *and* merge the new code to the current branch) from the added repo as you did before but choose the remote name of the repo you want to pull from (instead of `origin`):

Click the `Fetch` button or the `Pull` button first.



If the `Remote branch to pull` dropdown is empty, click the `Refresh` button on its right.

5. If the pull from the `samplerepo-things-2` was successful, you should have received one more commit into your local repo.

CLI

1. Navigate to the folder containing the local repo.

2. Set the new remote repo as a *remote* of the local repo.
   command: `git remote add {remote_name} {remote_repo_url}`
   e.g., `git remote add upstream1 https://github.com/johndoe/foobar.git`

3. Now you can fetch or pull (pulling will fetch the branch *and* merge the new code to the current branch) from the new remote.
   e.g., `git fetch upstream1 master` followed by `git merge upstream1/master` , or,
   `git pull upstream1 master`

Fork: Creating a remote copy

Given below is a scenario you can try in order to learn how to fork a repo:.

0. Create a GitHub account if you don't have one yet.

1. Go to the GitHub repo you want to fork e.g., samplerepo-things

2. Click on the 

button on the top-right corner. In the next step,

- choose to fork to your own account or to another GitHub organization that you are an admin of.
- Un-tick the `[ ] Copy the master branch only` option , so that you get copies of other branches (if any) in the repo.

As you might have guessed from the above, **forking is not a Git feature**, but a feature provided by remote Git hosting services such as Github.

GitHub does not allow you to fork the same repo more than once to the same destination. If you want to re-fork, you need to delete the previous fork.

`push` : Uploading data to other repos

Given below is a scenario you can try in order to learn how to push commits to a remote repo hosted on GitHub:

1. Fork an existing GitHub repo (e.g., samplerepo-things) to your GitHub account.

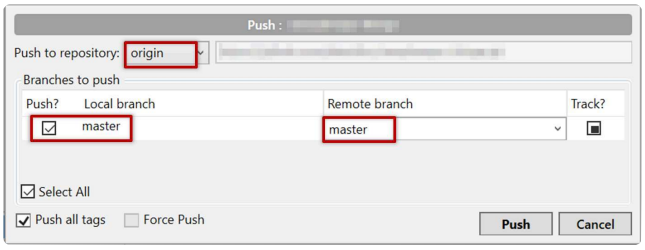
2. Clone the fork (not the original) to your computer.

3. Commit some changes in your local repo.

4. Push the new commits to your fork on GitHub

Sourcetree

Click the `Push` button on the main menu, ensure the settings are as follows in the next dialog, and click the `Push` button on the dialog.



CLI

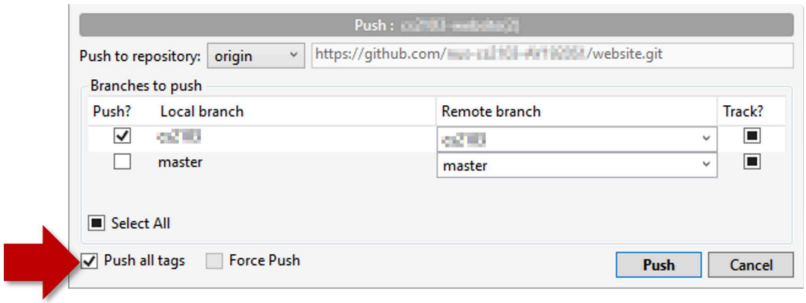
Use the command `git push origin master`. Enter your Github username and password when prompted.

5. Add a few more commits, and tag some of them.

6. Push the new commits *and* the tags.

Sourcetree

Push similar to before, but ensure the `[ ] Push all tags` option in the push dialog is ticked as well.



CLI

A normal push does not include tags. After pushing the commits (as before), push tags to the repo as well:

To push a specific tag:

```
$ git push origin v1.0b
```

To push all tags:

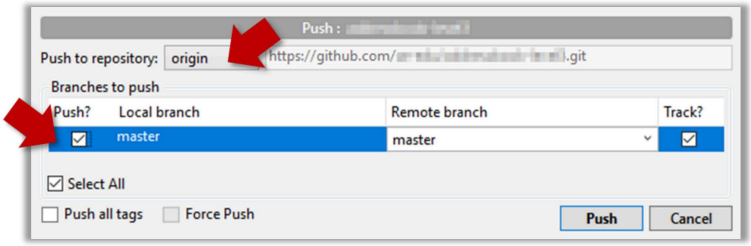
```
$ git push origin --tags
```

You can push to repos other than the one you cloned from, as long as the target repo and your repo have a shared history.

1. Add the GitHub repo URL as a remote, if you haven't done so already.
2. Push to the target repo.

Sourcetree

Push your repo to the new remote the usual way, but select the name of target remote instead of `origin` and remember to select the `Track` checkbox.



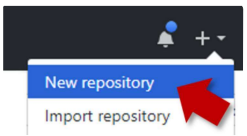
CLI

Push to the new remote the usual way e.g., `git push upstream1 master` (assuming you gave the name `upstream1` to the remote).

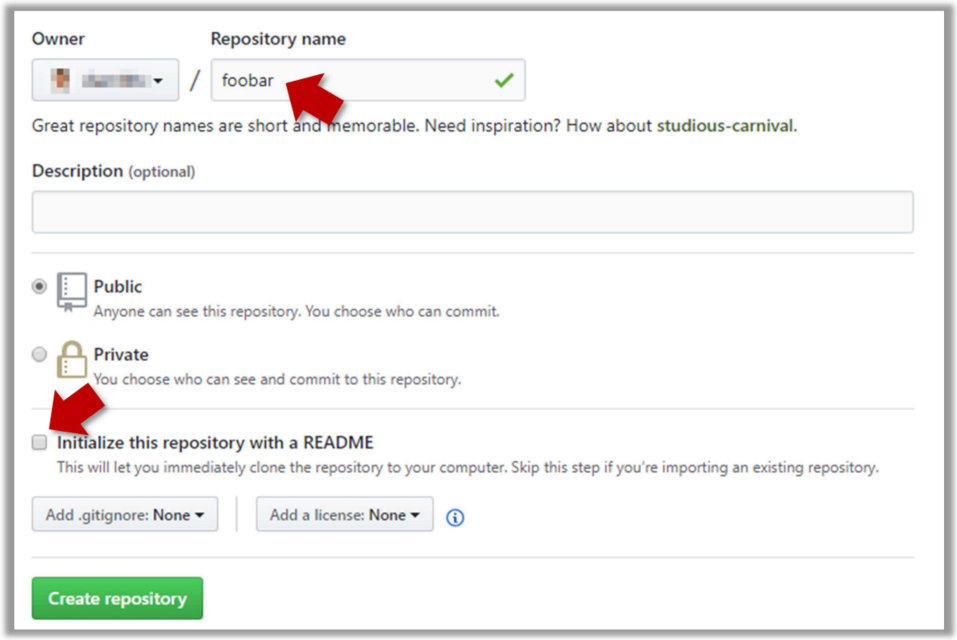
You can even push an entire local repository to GitHub, to form an entirely new remote repository. For example, you created a local repo and worked with it for a while but now you want to upload it onto GitHub (as a backup or to share it with others). The steps are given below.

1. Create an empty remote repo on GitHub.

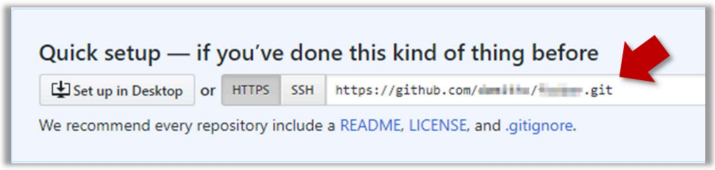
1. Login to your GitHub account and choose to create a new Repo.



2. In the next screen, provide a name for your repo but keep the `Initialize this repo ...` tick box unchecked.



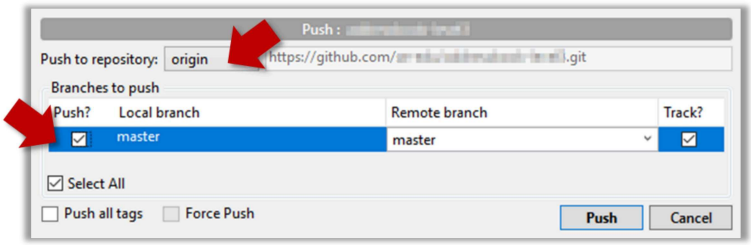
3. Note the URL of the repo. It will be of the form `https://github.com/{your_user_name}/{repo_name}.git` .
   e.g., `https://github.com/johndoe/foobar.git` (note the `.git` at the end)



2. Add the GitHub repo URL as a remote of the local repo. You can give it the name `origin` (or any other name).

3. Push the repo to the remote.

Sourcetree



CLI

Push each branch to the new remote the usual way but use the `-u` flag to inform Git that you wish to track the branch.
e.g., `git push -u origin master`

`branch` : Doing multiple parallel changes ⚡⚡

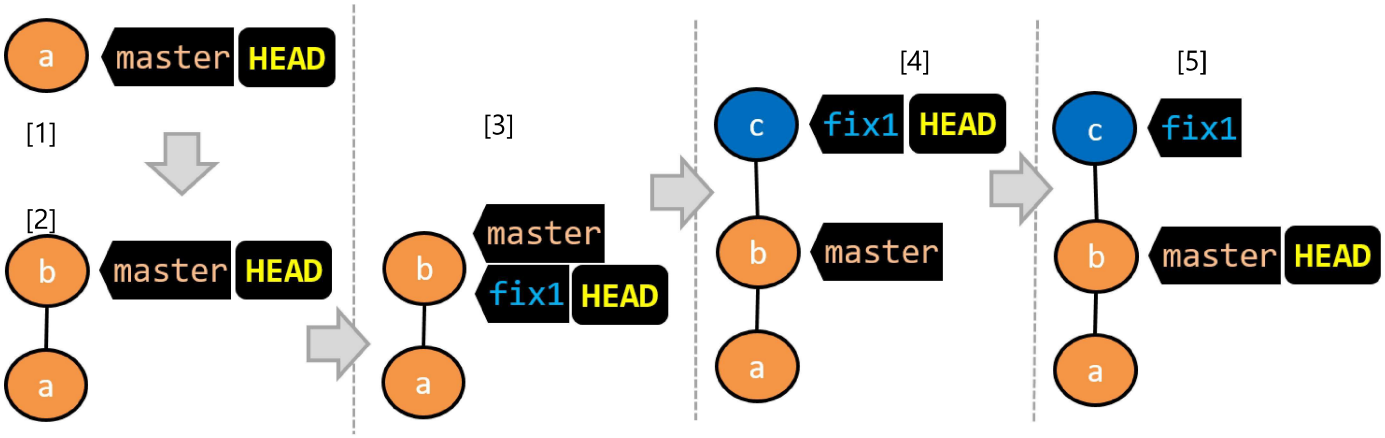
Git supports branching, which allows you to do multiple parallel changes to the content of a repository.

First, let us learn how the repo looks like as you perform branching operations.

A Git branch is simply a *named label* pointing to a commit which moves forward automatically as you add more commits to that branch. **The `HEAD` label indicates which branch you are on.**

Git creates a branch named `master` (or `main`) by default. When you add a commit, it goes into the branch you are currently on, and the branch label (together with the `HEAD` label) moves to the new commit.

Given below is an illustration of how branch labels move as branches evolve. Refer to the text below it for explanations of each stage.



1. There is only one branch (i.e., `master`) and there is only one commit on it. The `HEAD` label is pointing to the `master` branch (as we are currently on that branch).

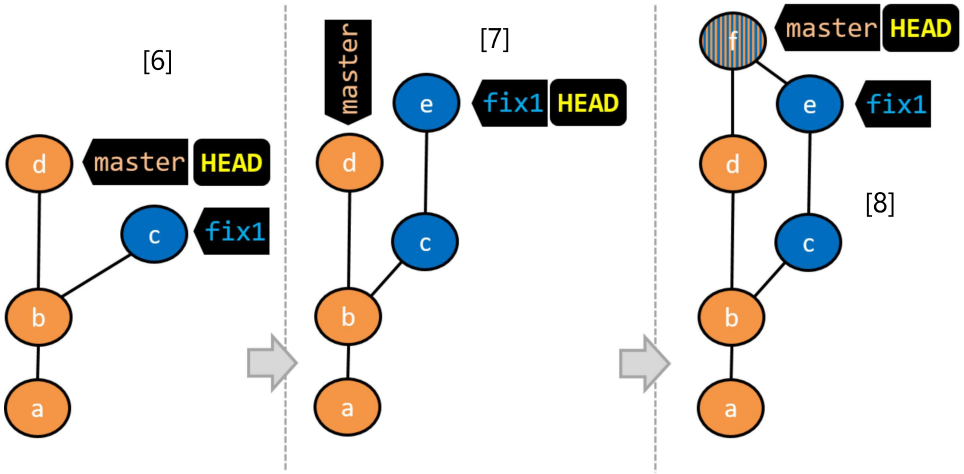
   ℹ To learn a bit more about how labels such as `master` and `HEAD` work, you can refer to this article.

2. A new commit has been added. The `master` and the `HEAD` labels have moved to the new commit.

3. A new branch `fix1` has been added. The repo has switched to the new branch too (hence, the `HEAD` label is attached to the `fix1` branch).

4. A new commit (`c`) has been added. The current branch label `fix1` moves to the new commit, together with the `HEAD` label.

5. The repo has switched back to the `master` branch. Hence, the `HEAD` has moved back to `master` branch's tip.
   At this point, the repo's working directory reflects the code at commit `b` (not `c`).



6. A new commit (`d`) has been added. The `master` and the `HEAD` labels have moved to that commit.
7. The repo has switched back to the `fix1` branch and added a new commit (`e`) to it.

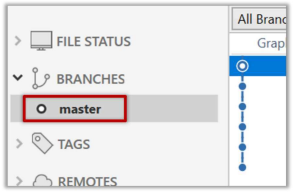
8. The repo has switched to the `master` branch and the `fix1` branch has been merged into the `master` branch, creating a *merge commit* `f`. The repo is currently on the `master` branch.

Now that you have some idea how the repo will look like when branches are being used, let's follow the steps below to learn how to perform branching operations using Git. You can use any repo you have on your computer (e.g. a clone of the samplerepo-things) for this.

! Note that how the revision graph appears (colors, positioning, orientation etc.) in your Git GUI varies based on the GUI you use, and might not match the exact diagrams given above.

0. Observe that you are normally in the branch called `master`.

Sourcetree



CLI

```
$ git status
```

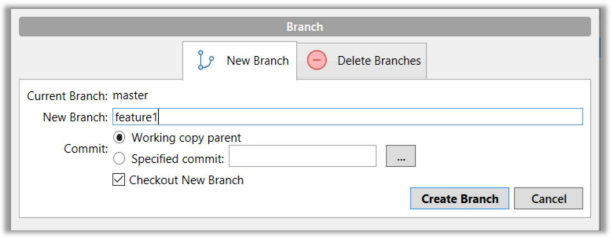
↓

```
on branch master
```

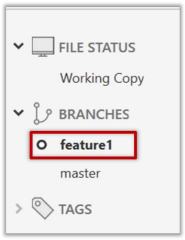
1. Start a branch named `feature1` and switch to the new branch.

Sourcetree

Click on the `Branch` button on the main menu. In the next dialog, enter the branch name and click `Create Branch`.



Note how the `feature1` is indicated as the current branch (reason: Sourcetree automatically switches to the new branch when you create a new branch).



CLI

You can use the `branch` command to create a new branch and the `checkout` command to switch to a specific branch.

```
$ git branch feature1
$ git checkout feature1
```

One-step shortcut to create a branch and switch to it at the same time:

```
$ git checkout -b feature1
```

The new `switch` command

Git recently introduced a `switch` command that you can use instead of the `checkout` command given above.

To create a new branch and switch to it:

```
$ git branch feature1
$ git switch feature1
```

One-step shortcut:

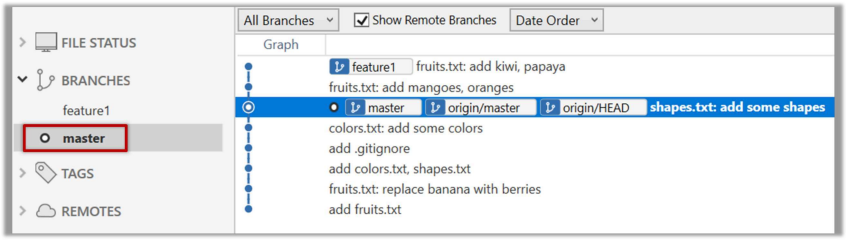
```
$ git switch -c feature1
```

2. Create some commits in the new branch. Just commit as per normal. Commits you add while on a certain branch will become part of that branch. Note how the `master` label and the `HEAD` label moves to the new commit (The `HEAD` label of the local repo is represented as ● in Sourcetree, as illustrated in the screenshot below).

3. Switch to the `master` branch. Note how the changes you did in the `feature1` branch are no longer in the working directory.

Sourcetree

Double-click the `master` branch.



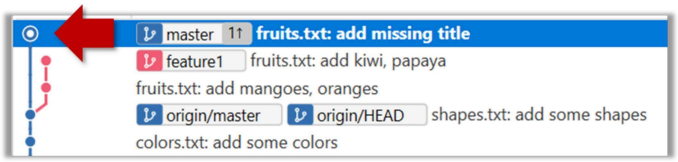
i **Revisiting `master` vs `origin/master`**

In the screenshot above, you see a `master` label and a `origin/master` label for the same commit. The former identifies the tip of the local `master` branch while the latter identifies the tip of the `master` branch at the remote repo named `origin`. The fact that both labels point to the same commit means the local `master` branch and its remote counterpart are in sync with each other. Similarly, `origin/HEAD` label appearing against the same commit indicates that the `HEAD` label of the remote repo is pointing to this commit as well.

CLI

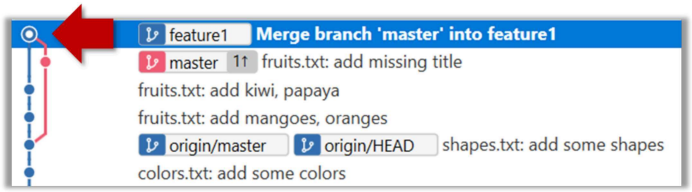
```
$ git checkout master
```

4. Add a commit to the master branch. Let's imagine it's a bug fix.
To keep things simple for the time being, this commit should not involve the same content that you changed in the `feature1` branch. To be on the safe side, you can change an entirely different file in this commit.



5. Switch back to the `feature1` branch (similar to step 3).

6. Merge the `master` branch to the `feature1` branch, giving an end-result like the following. Also note how Git has created a *merge commit*.



Sourcetree

Right-click on the `master` branch and choose `merge master into the current branch`. Click `OK` in the next dialog.

CLI

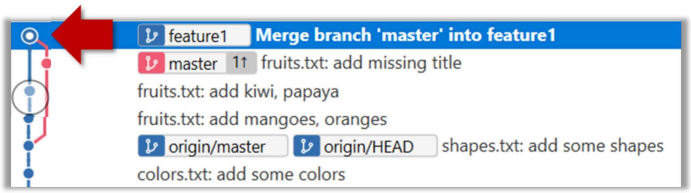
```
$ git merge master
```

The objective of that merge was to *sync* the `feature1` branch with the `master` branch. Observe how the changes you did in the `master` branch (i.e. the imaginary bug fix) is now available even when you are in the `feature1` branch.

To undo a merge,

1. Ensure you are in the branch that received the merge.
2. Do a hard reset (similar to how you delete a commit) of that branch to the commit that would be the tip of that branch had you not done the offending merge.

📦 In the example below, you merged `master` to `feature1`.



If you want to undo that merge,

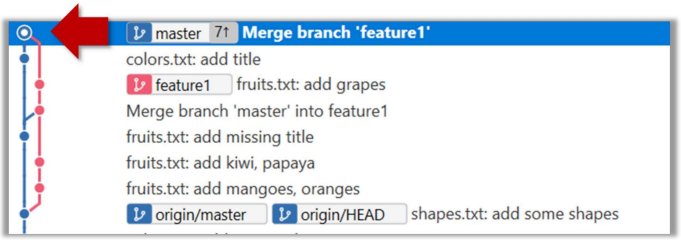
1. Ensure you are in the `feature1` branch (because that's the branch that *received* the merge).
2. Reset the `feature1` branch to the commit highlighted (in yellow) in the screenshot above (because that was the tip of the `feature1` branch before you merged the `master` branch to it.

ℹ Instead of merging `master` to `feature1`, an alternative is to *rebase* the `feature1` branch. However, rebasing is an advanced feature that requires modifying past commits. If you modify past commits that have been pushed to a remote repository, you'll have to *force-push* the modified commit to the remote repo in order to update the commits in it.

7. Add another commit to the `feature1` branch.

8. Switch to the `master` branch and add one more commit.

9. Merge `feature1` to the master branch, giving and end-result like this:



Sourcetree

Right-click on the `feature1` branch and choose `Merge...`.

CLI

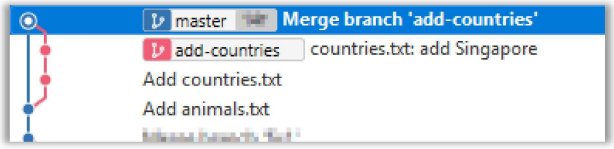
```
$ git merge feature1
```

10. Create a new branch called `add-countries`, switch to it, and add some commits to it (similar to steps 1-2 above). You should have something like this now:

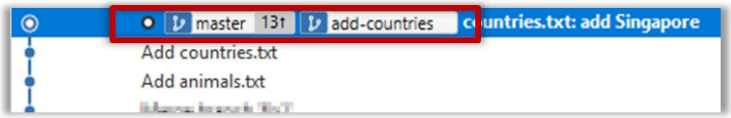
✕ **Avoid this rookie mistake!**

Always remember to switch back to the `master` branch before creating a new branch. If not, your new branch will be created on top of the current branch.

11. Go back to the `master` branch and merge the `add-countries` branch onto the `master` branch (similar to steps 8-9 above). While you might expect to see something like the following,



... you are likely to see something like this instead:

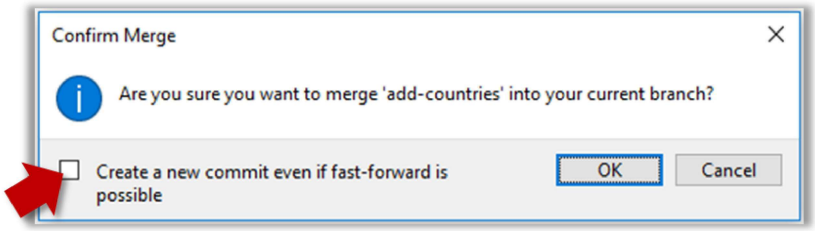


That is because **Git does a *fast forward* merge if possible**. Seeing that the `master` branch has not changed since you started the `add-countries` branch, Git has decided it is simpler to just put the commits of the `add-countries` branch in front of the `master` branch, without going into the trouble of creating an extra merge commit.

It is possible to force Git to create a merge commit even if fast forwarding is possible.

Sourcetree

Windows: Tick the box shown below when you merge a branch:



 Mac:

1. Go to Sourcetree `Settings`.
2. Navigate to the `Git` section.
3. Tick the box `Do not fast-forward when merging, always create commit`.

CLI

Use the `--no-ff` switch (short for *no fast forward*):

```
$ git merge --no-ff add-countries
```

Dealing with merge conflicts ⚡⚡

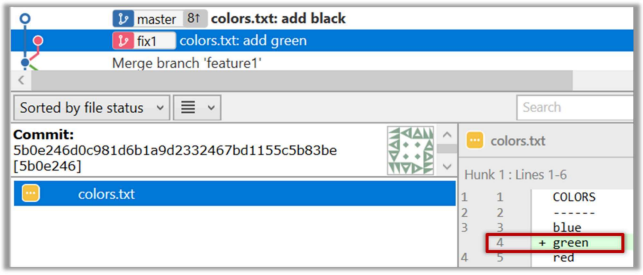
Merge conflicts happen when you try to combine two incompatible versions (e.g., merging a branch to another but each branch changed the same part of the code in a different way).

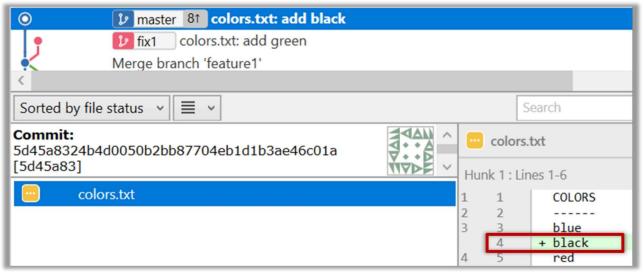
Here are the steps to simulate a merge conflict and use it to learn how to resolve merge conflicts.

0. Create an empty repo or clone an existing repo, to be used for this activity.

1. Start a branch named `fix1` in the repo. Create a commit that adds a line with some text to one of the files.

2. Switch back to `master` branch. Create a commit with a conflicting change i.e. it adds a line with some different text in the exact location the previous line was added.





3. Try to merge the `fix1` branch onto the `master` branch. Git will pause mid-way during the merge and report a merge conflict. If you open the conflicted file, you will see something like this:

```
COLORS
------
blue
<<<<<< HEAD
black
=======
green
>>>>>> fix1
red
white
```

4. Observe how the conflicted part is marked between a line starting with `<<<<<<` and a line starting with `>>>>>>`, separated by another line starting with `=======`.

Highlighted below is the conflicting part that is coming from the `master` branch:

```
blue
<<<<<< HEAD
black
=======
green
>>>>>> fix1
red
```

This is the conflicting part that is coming from the `fix1` branch:

```
blue
<<<<<< HEAD
black
=======
green
>>>>>> fix1
red
```

5. Resolve the conflict by editing the file. Let us assume you want to keep both lines in the merged version. You can modify the file to be like this:

```
COLORS
------
blue
black
green
red
white
```

6. Stage the changes, and commit. You have now successfully resolved the merge conflict.

Remote branches ⚡⚡

Git branches in a local repo can be *linked* to a branch in a remote repo so the local branch can 'track' the corresponding remote branch, and revision history contained in the local and the remote branch pair can be synchronized as desired.

[A] Pushing a new branch to a remote repo

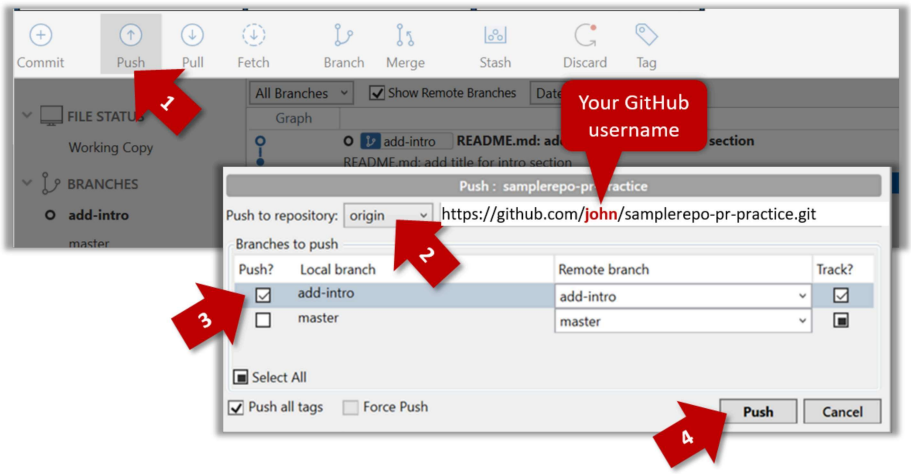
Let's see how you can push a branch that you created in your local repo to the remote repo. Note that this branch does not exist in the remote repo yet.

Given below is how to push a branch named `add-intro` to your own fork named `samplerepo-pr-practice`.

We assume that your local repo already has the remote added to it with the name `origin`. If that is not the case, you should first configure your local repo to be able to communicate with the target remote repo.

Sourcetree

1. Click on `Push` button, which opens up the Push dialog.
2. Choose the remote that you wish to push the branch (assuming you've added that repo to your Sourcetree already).
3. Select the branch(es) you want to push -- in this case, `add-intro` .
   Ensure the `Track?` checkbox is ticked for the selected branch(es).
4. Click `Push` .



CLI

```
$ git push -u origin add-intro
```

The `-u` (or `--set-upstream`) flag tells Git that you wish the local branch to 'track' the remote branch that will be created as a result of this push.

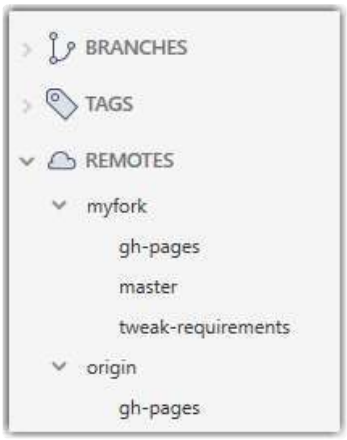
See git-scm.com/docs/git-push for details of the `push` command.

[B] Pulling a remote branch for the first time

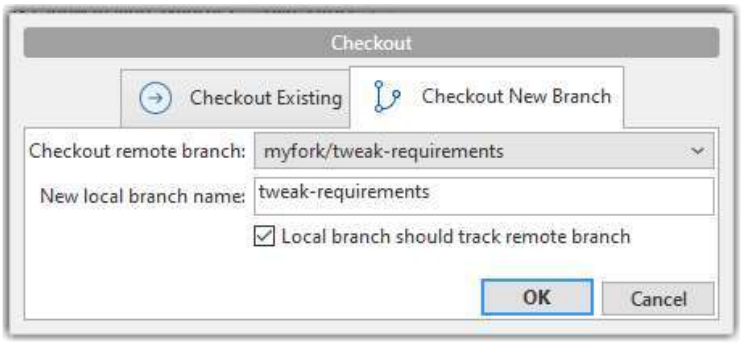
Here, let's see how to fetch a new branch (i.e., it does not exist in your local repo yet) from a remote repo.

Sourcetree

1. Check the list of remote branches by expanding the `REMOTES` menu on the left edge of Sourcetree. If the branch you expected to find is missing, you can click the `Fetch` button (in the top toolbar) to refresh the information shown under remotes.



2. Double-click the branch name (e.g., `tweak-requirements` branch in the `myfork` remote), which should open the checkout dialog shown below.



3. Go with the default settings (shown above) should be fine. Once you click `OK`, the branch will appear in your local repo. Furthermore, that repo will switch to that branch, and the local branch will track the remote branch as well.

CLI

1. Fetch details from the remote. e.g., if the remote is named `myfork`

```
$ git fetch myfork
```

2. List the branches to see the name of the branch you want to pull.

```
$ git branch -a
```

↓

```
master
remotes/myfork/master
remotes/myfork/branch1
```

`-a` flag tells Git to list both local and remote branches.

3. Create a matching local branch and switch to it.

```
$ git switch -c branch1 myfork/branch1
```

↓

```
Switched to a new branch 'branch1'
```

```
branch 'branch1' set up to track 'myfork/branch1'.
```

`-c` flag tells Git to create a new local branch.

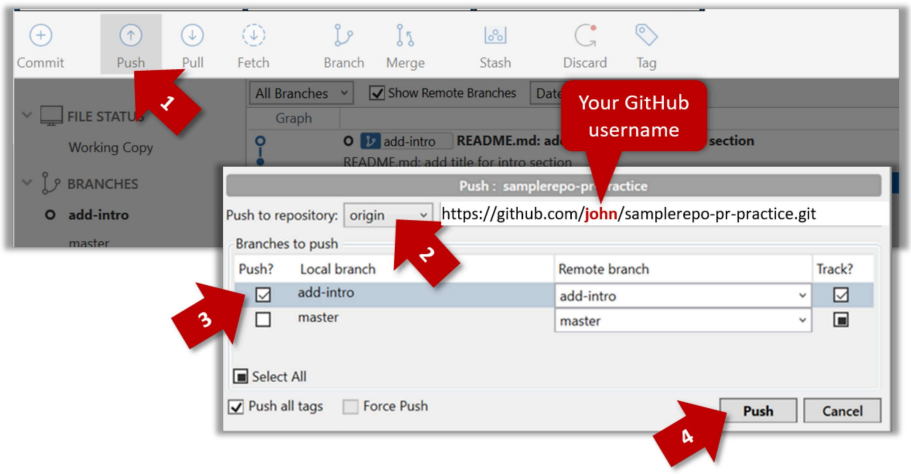
[C] Syncing branches

In this section we assume that you have a local branch that is already tracking a remote branch (e.g., as a result of doing [A] or [B] above).

[C1] To push new changes in the local branch to the corresponding remote branch:

Sourcetree

Similar to how you pushed a new branch (in [A]):



CLI

Similar to [A] above, but omit the `-u` flag. e.g.,

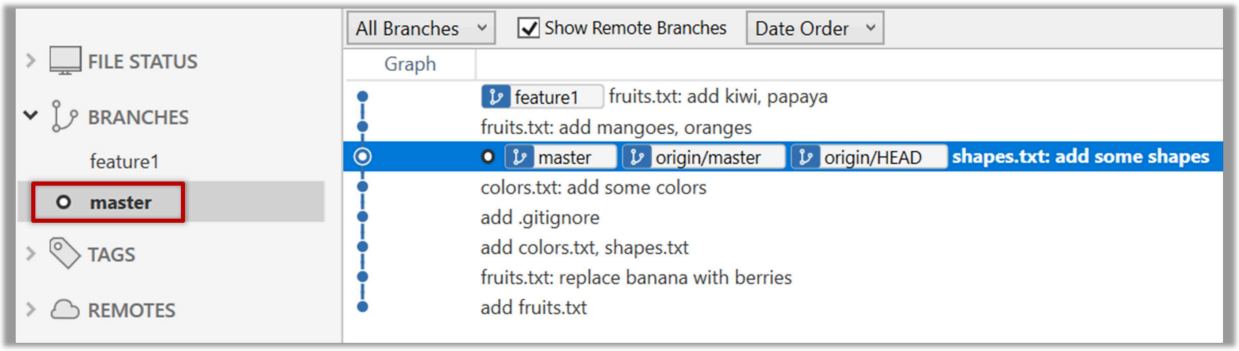
```
$ git push origin add-intro
```

ℹ **If you push but the remote branch has new commits that you don't have locally**, Git will abort the push and will ask you to pull first.

[C2] To pull new changes from a remote branch to the corresponding local branch:

Sourcetree

1. Switch to the branch you want to update by double-clicking the branch name. e.g.,



2. Pull the updated in the remote branch to the local branch by right-clicking on the branch name (in the same place as above), and choosing `Pull <remote>/<branch> (tracked)` e.g., `Pull myfork/add-intro (tracked)` .

CLI

1. Switch to the branch you want to update using `git checkout <branch>` e.g.,

```
$ git checkout branch1
```

2. Pull the updated in the remote branch to the local branch, using `git pull <remote> <branch>` e.g.,

```
$ git pull origin branch1
```

i **If you pull but your local branch has new commits the remote branch doesn't have**, Git will automatically perform a merge between the local branch and the remote branch.

Creating PRs

Suppose you want to propose some changes to a GitHub repo (e.g., samplerepo-pr-practice) as a pull request (PR).

samplerepo-pr-practice is an unmonitored repo you can use to practice working with PRs. Feel free to send PRs to it.

Given below is a scenario you can try in order to learn how to create PRs:

1. Fork the repo onto your GitHub account.

2. Clone it onto your computer.

3. Commit your changes e.g., add a new file with some contents and commit it.

- **Option A - Commit changes to the `master` branch**
- **Option B - Commit to a new branch** e.g., create a branch named `add-intro` (remember to switch to the `master` branch before creating a new branch) and add your commit to it.

4. Push the branch you updated (i.e., `master` branch or the new branch) to your fork, as explained here.

5. Initiate the PR creation:

1. Go to your fork.

2. Click on the `Pull requests` tab followed by the New pull request **button**. This will bring you to the `Compare changes` page.

3. Set the appropriate target repo and the branch that should receive your PR, using the `base repository` and `base` dropdowns. e.g.,
   base repository: **se-edu/samplerepo-pr-practice** base: **master**

   > ℹ Normally, the default value shown in the dropdown is what you want but in case your fork has multiple upstream repos, the default may not be what you want.

4. Indicate which repo:branch contains your proposed code, using the `head repository` and `compare` dropdowns. e.g.,
   head repository: **myrepo/samplerepo-pr-practice** compare: **master**

6. Verify the proposed code: Verify that the diff view in the page shows the exact change you intend to propose. If it doesn't, update the branch as necessary.

7. Submit the PR:

1. Click the Create pull request button.

2. Fill in the PR name and description e.g.,
   Name: `Add an introduction to the README.md`
   Description:

   ```
   Add some paragraph to the README.md to explain ...
   Also add a heading ...
   ```

3. If you want to indicate that the PR you are about to create is 'still work in progress, not yet ready', click on the dropdown arrow in the Create pull request button and choose `Create draft pull request` option.

4. Click the Create pull request button to create the PR.

5. Go to the receiving repo to verify that your PR appears there in the `Pull requests` tab.

The next step of the PR lifecycle is the PR *review*. The members of the repo that received your PR can now review your proposed changes.

- If they like the changes, they can *merge* the changes to their repo, which also closes the PR automatically.
- If they don't like it at all, they can simply close the PR too i.e., they reject your proposed change.
- In most cases, they will add comments to the PR to suggest further changes. When that happens, GitHub will notify you.

You can update the PR along the way too. Suppose PR reviewers suggested a certain improvement to your proposed code. To update your PR as per the suggestion, you can simply modify the code in your local repo, commit the updated code to the same branch as before, and push to your fork as you did earlier. The PR will auto-update accordingly.

Sending PRs using the `master` branch is less common than sending PRs using separate branches. For example, suppose you wanted to propose two bug fixes that are not related to each other. In that case, it is more appropriate to send two separate PRs so that each fix can be reviewed, refined, and merged independently. But if you send PRs using the `master` branch only, both fixes (and any other change you do in the `master` branch) will appear in the PRs you create from it.

To create another PR while the current PR is still under review, create a new branch (remember to switch back to the `master` branch first), add your new proposed change in that branch, and create a new PR following the steps given above.

It is possible to create PRs within the same repo e.g., you can create a PR from branch `feature-x` to the `master` branch, within the same repo. Doing so will allow the code to be reviewed by other developers (using PR review mechanism) before it is merged.

Problem: merge conflicts in ongoing PRs, indicated by the message **This branch has conflicts that must be resolved**. That means the upstream repo's `master` branch has been updated in a way that the PR code conflicts with that `master` branch. Here is the standard way to fix this problem:

1. Pull the `master` branch from the upstream repo to your local repo.

```
git checkout master
git pull upstream master
```

2. In the local repo, attempt to merge the `master` branch (that you updated in the previous step) onto the PR branch, in order to bring over the new code in the `master` branch to your PR branch.

```
git checkout pr-branch  # assuming pr-branch is the name of branch in the PR
git merge master
```

3. The merge you are attempting will run into a merge conflict, due to the aforementioned conflicting code in the `master` branch. Resolve the conflict manually (this topic is covered elsewhere), and complete the merge.
4. Push the PR branch to your fork. As the updated code in that branch no longer is conflicting with the `master` branch, the merge conflict alert in the PR will go away automatically.

Reviewing PRs ⚡⚡

The *PR review* stage is a dialog between the PR author and members of the repo that received the PR, in order to refine and eventually merge the PR.

Given below are some steps you can follow when reviewing a PR.

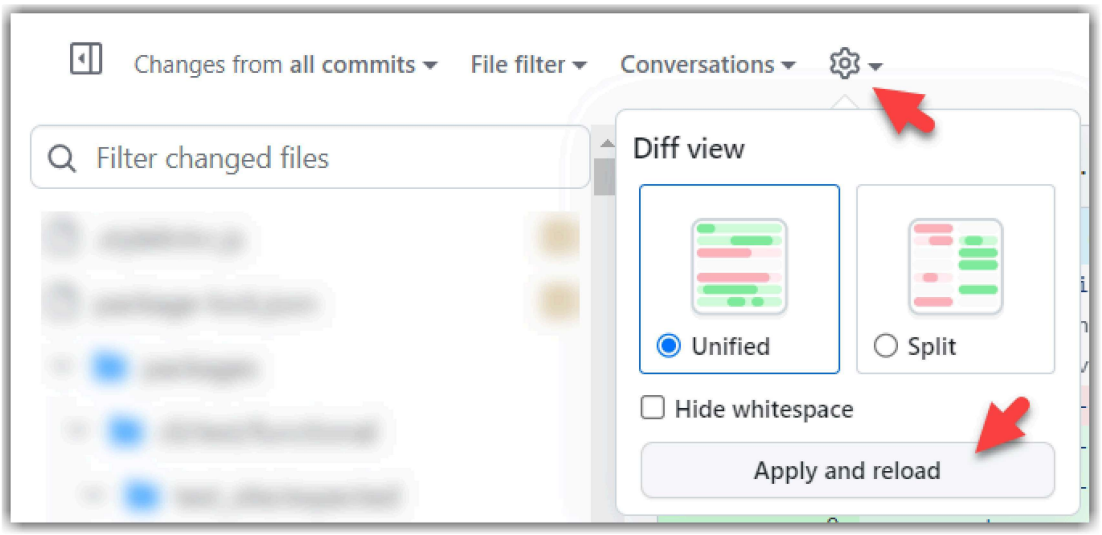
1. Locate the PR:

1. Go to the GitHub page of the repo.
2. Click on the `Pull requests` tab.
3. Click on the PR you want to review.

2. Read the PR description. It might contain information relevant to reviewing the PR.

3. Click on the `Files changed` tab to see the *diff* view.

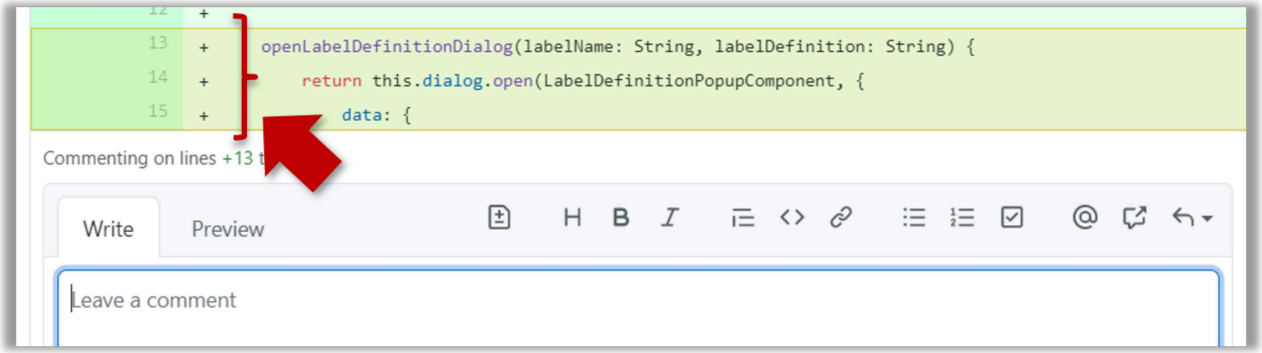
- You can use the following setting to try the two different views available and pick the one you like.



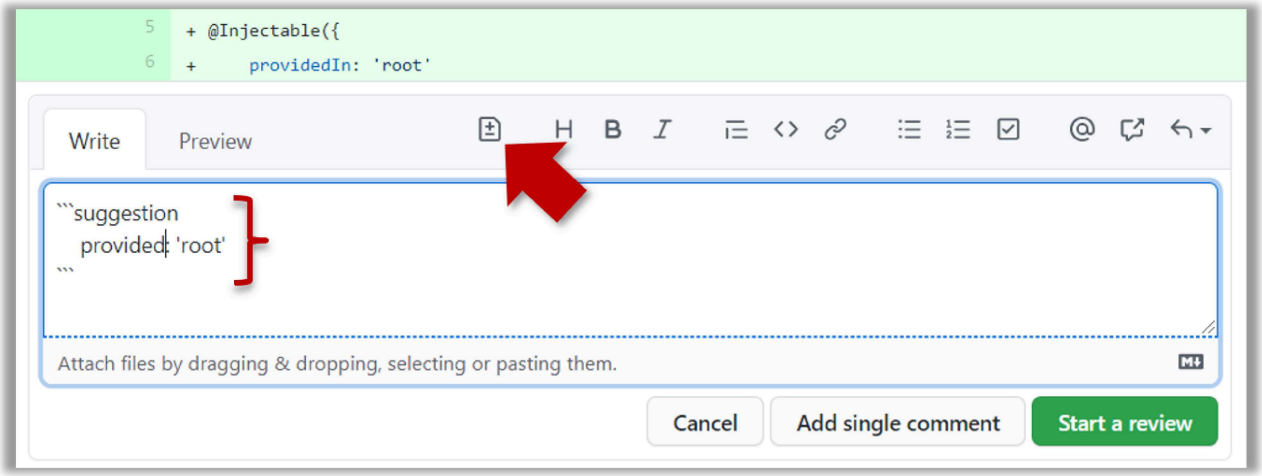
4. Add review comments:

1. Hover over the line you want to comment on and click on the ➕ icon that appears on the left margin. That should create a text box for you to enter your comment.

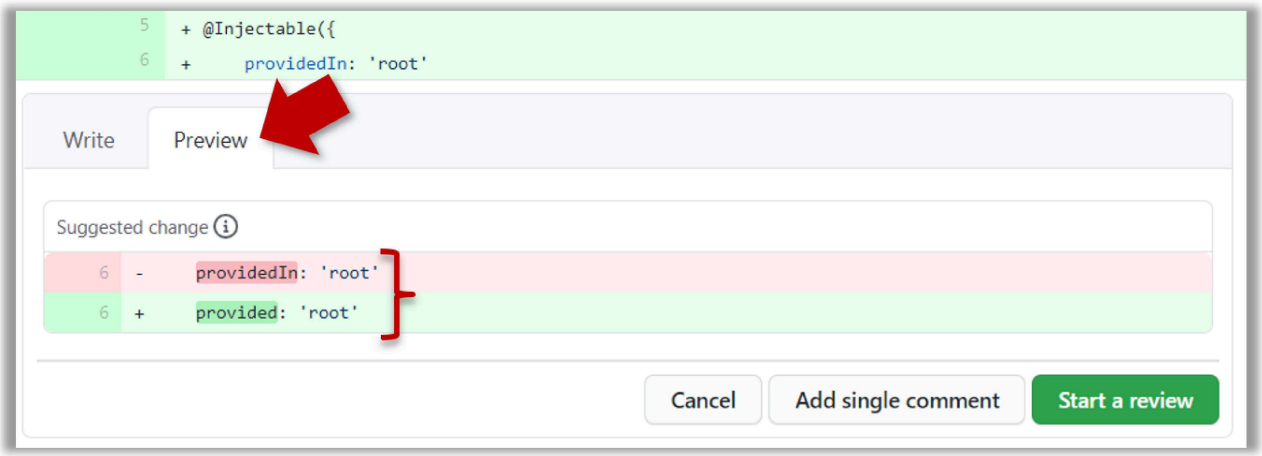
- To give a comment related to multiple lines, click-and-drag the ➕ icon. The result will look like this:



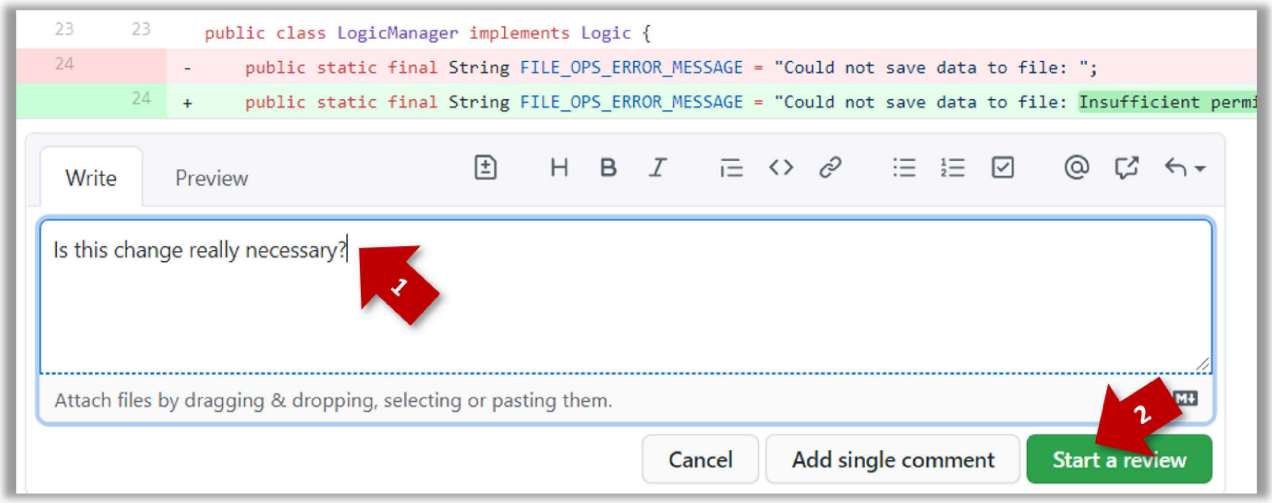
2. Enter your comment.
   - This page @SE-EDU/guides has some best practices PR reviewers can follow.
   - To suggest an in-line code change, click on this icon:



   After that, you can proceed to edit the `suggestion` code block generated by GitHub (as seen in the screenshot above).
   The comment will look like this to the viewers:

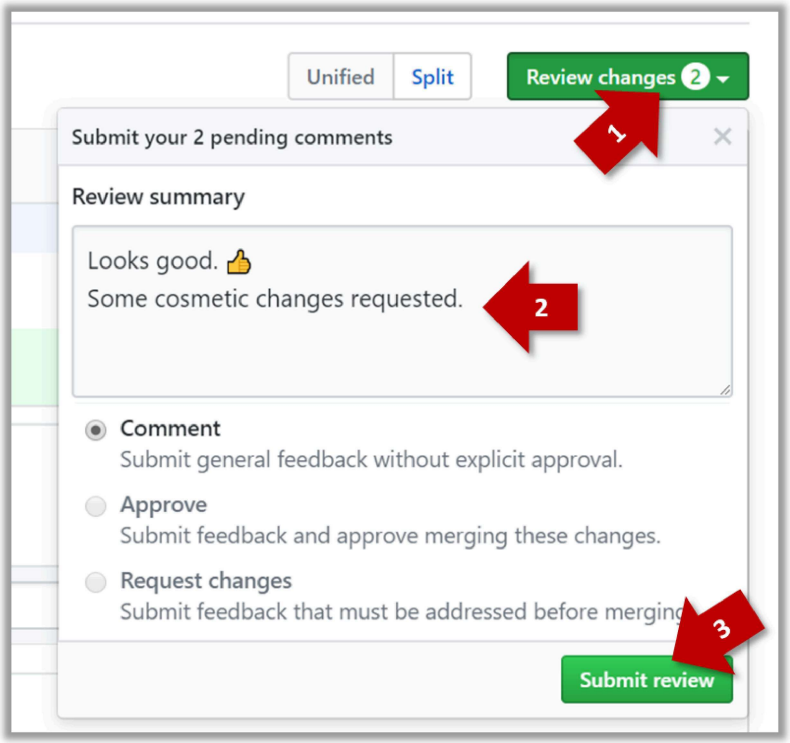


3. After typing in the comment, click on the `Start a review` button (not the `Add single comment` button. This way, your comment is saved but not visible to others yet. It will be visible to others only when you have finished the entire review.



4. Repeat the above steps to add more comments.

5. Submit the review:



1. When there are no more comments to add, click on the `Review changes` button (on the top right of the diff page).
2. Type in an overall comment about the PR, if any. e.g.,

```
Overall, I found your code easy to read for the most part except a few places
where the nesting was too deep. I noted a few minor coding standard violations
too. Some of the classes are getting quite long. Consider splitting into
smaller classes if that makes sense.
```

   `LGTM` is often used in such overall comments, to indicate `Looks good to me` (or `Looks good to merge`). `nit` (as in *nit-picking*) is another such term, used to indicate minor flaws e.g., `LGTM. Just a few nits to fix.`.
3. Choose `Approve`, `Comment`, or `Request changes` option as appropriate and click on the `Submit review` button.

Merging PRs ⚡⚡

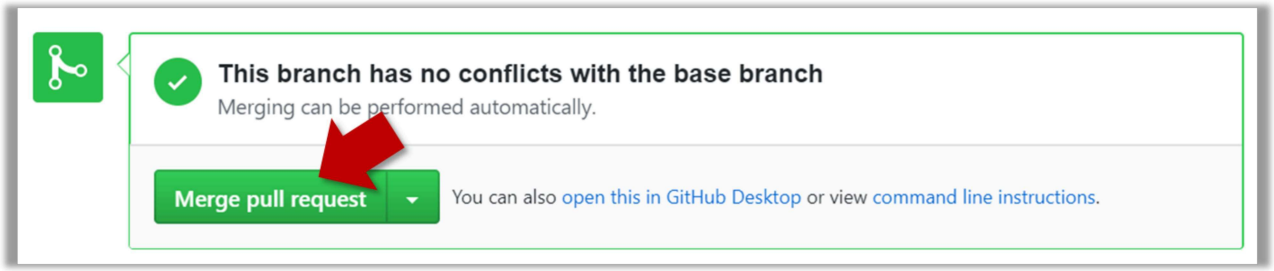
Let's look at the steps involved in merging a PR, assuming the PR has been reviewed, refined, and approved for merging already.

Preparation: If you would like to try merging a PR yourself, you can create a dummy PR in the following manner.

1. Fork any repo (e.g., samplerepo-pr-practice).
2. Clone in to your computer.
3. Create a new branch e.g., (`feature1`) and add some commits to it.
4. Push the new branch to the fork.
5. Create a PR from that branch to the `master` branch in your fork. Yes, it is possible to create a PR within the same repo.

1. Locate the PR to be merged in your repo's GitHub page.

2. Click on the `Conversation` tab and scroll to the bottom. You'll see a panel containing the PR status summary.



3. If the PR is not merge-able in the current state, the Merge pull request will not be green. Here are the possible reasons and remedies:

- **Problem: The PR code is out-of-date**, indicated by the message **This branch is out-of-date with the base branch**. That means the repo's `master` branch has been updated since the PR code was last updated.
  - If the PR author has allowed you to update the PR and you have sufficient permissions, GitHub will allow you to update the PR simply by clicking the Update branch on the right side of the 'out-of-date' error message. If that option is not available, post a message in the PR requesting the PR author to update the PR.
- **Problem: There are merge conflicts**, indicated by the message **This branch has conflicts that must be resolved**. That means the repo's `master` branch has been updated since the PR code was last updated, in a way that the PR code conflicts with the current `master` branch. Those conflicts must be resolved before the PR can be merged.
  - If the conflicts are simple, GitHub might allow you to resolve them using the Web interface.
  - If that option is not available, post a message in the PR requesting the PR author to update the PR.

4. Merge the PR by clicking on the Merge pull request button, followed by the `Confirm merge` button. You should see a `Pull request successfully merged and closed` message after the PR is merged.

- You can choose between three merging options by clicking on the down-arrow in the Merge pull request button. If you are new to Git and GitHub, the `Create merge commit` option is recommended.

Next, sync your local repos (and forks). Merging a PR simply merges the code in the upstream remote repository in which it was merged. The PR author (and other members of the repo) needs to pull the merged code from the upstream repo to their local repos and push the new code to their respective forks to sync the fork with the upstream repo.

Forking Workflow ⚡⚡⚡

You can follow the steps in the simulation of a forking workflow given below to learn how to follow such a workflow.

ℹ️ This activity is best done as a team.

Step 1. One member: set up the team org and the team repo.

1.1) **Create a GitHub organization** for your team, if you don't have one already. The org name is up to you. We'll refer to this organization as *team org* from now on.

1.2) **Add a team** called `developers` to your team org.

1.3) **Add team members** to the `developers` team.

1.4) **Fork** se-edu/samplerepo-workflow-practice to your team org. We'll refer to this as the *team repo*.

1.5) **Add the forked repo to the `developers` team.** Give write access.

Step 2. Each team member: create PRs via own fork.

2.1) **Fork that repo** from your team org to your own GitHub account.

2.2) **Create a branch** named `add-{your name}-info` (e.g. `add-johnTan-info`) in the local repo.

2.3) **Add a file** `yourName.md` into the `members` directory (e.g., `members/johnTan.md`) containing some info about you into that branch.

2.4) **Push that branch to your fork**.

2.5) **Create a PR** from that branch to the `master` branch of the team repo.

Step 3. For each PR: review, update, and merge.

3.1) **[A team member (not the PR author)] Review the PR** by adding comments (can be just dummy comments).

3.2) **[PR author] Update the PR** by pushing more commits to it, to simulate updating the PR based on review comments.

3.3) **[Another team member] Approve and merge** the PR using the GitHub interface.

3.4) **[All members] Sync your local repo (and your fork)** with upstream repo. In this case, your *upstream repo* is the repo in your team org.

- The basic mechanism for this has two steps (which you can do using Git CLI or any Git GUI):
  (1) First, pull from the upstream repo -- this will update your clone with the latest code from the upstream repo.
  If there are any unmerged branches in your local repo, you can update them too e.g., you can merge the new `master` branch to each of them.
  (2) Then, push the updated branches to your fork. This will also update any PRs from your fork to the upstream repo.
- Some alternatives mechanisms to achieve the same can be found in this GitHub help page.
  If you are new to Git, we recommend that you use the above two-step mechanism instead, so that you get a better view of what's actually happening behind the scene.

Step 4. Create conflicting PRs.

4.1) **[One member]: Update README:** In the `master` branch, remove John Doe and Jane Doe from the `README.md`, commit, and push to the main repo.

4.2) **[Each team member] Create a PR** to add yourself under the `Team Members` section in the `README.md`. Use a new branch for the PR e.g., `add-johnTan-name`.

Step 5. Merge conflicting PRs one at a time. Before merging a PR, you'll have to resolve conflicts.

5.1) [Optional] A member can inform the PR author (by posting a comment) that there is a conflict in the PR.

5.2) **[PR author] Resolve the conflict locally**:

1. Pull the `master` branch from the repo in your team org.
2. Merge the pulled `master` branch to your PR branch.
3. Resolve the merge conflict that crops up during the merge.
4. Push the updated PR branch to your fork.

5.3) **[Another member or the PR author]: Merge the de-conflicted PR**: When GitHub does not indicate a conflict anymore, you can go ahead and merge the PR.